Tylersoft Card Processor

API v.2.1

Revision History

Version	Author	Date	Description
1.0	Gaonyadiwe Gaboiphiwe	2025-05-08	Initial draft

Plot 75782, Pinnacle Park, Setlhoa Village, Gaborone | Private Bag BO 70, Gaborone, Botswana|
Tel: +(267) 3951798 | Fax: +(267) 3951898 | Email: enquiries@tylersoft.net

Introduction

The Tylesoft Payment API provides functionality for payment processing for authorization capture and settlement of card transactions worldwide. You can use the API documentation to integrate into our different endpoints. This guide is written for application developers who want to use Tylersoft's Card Processor (TCP) to integrate to their relevant systems for card payment.

Purpose

This guide describes the tasks you must complete on your application in order to be able to send a successful request to TCP.

Requirements

For your organization to be setup in TCP, Tylersoft will require the following

a. Organization email

b. Post Back url

Authkey Generation

This simply adds a level of security to ensure that only an authorized organization is able to send a transaction to TCP.

The authkey submitted for transaction processing should be constructed using the following format

Sha512(base64(key+tranid+amount+timestamp+phone))

Where;

Key – unique organization key (this is given by Tylersoft)

Tranid – unique transaction id

Amount – the amount to be debited

Timestamp – transaction request time

Phone – card holders phone number

Example If

key = !@#$%^&*()~_+}{?

Tranid =1919091

Amount=3

Timestamp =2017-01-22 03:41:02+0000

Phone = 254721765451

Then authkey = **Sha512(base64(**!@#$%^&*()~_+}{?191909132017-01-22 03:41:02+0000254721765451**))**
The base64 of **string(**!@#$%^&*()~_+}{?191909132017-01-22 03:41:02+0000254721765451**)** should be

IUAjJCVeJiooKX5fK317PzE5MTkwOTEzMjAxNy0wMS0yMiAwMzo0MTowMiswMDAwMjU0NzIxNzY1NDUx

The Sha512 of **base64 (IUAjJCVeJiooKX5fK317PzE5MTkwOTEzMjAxNy0wMS0yMiAwMzo0MTowMiswMDAwMjU0NzIxNzY1NDUx)**
Should be

0105FC3963F29C1DCA74229D499E79CD9DB0D0FDF32B203676FD802CC9191727FE3166A80D95DD8893512531F37E59FB7DA853849DFA9DAAFC4AD49821C71529

Note: The control key should not be included anywhere on the json request or any other form of submission to TCP. This would compromise the security of the control key and could possibly allow for outsiders to view or capture the control

Device Finger-Print (Device Data Collection)

This API functionality is used for collecting information about the device that the customer is using to transact. TCP will respond with a device collection url and access token that the client would use to send device data to the issuing bank.

Sample Request

```
{
   "tranid": "2025-05-01 09:25:41",
   "country": "Botswana",
   "amount": "1",
```

"authkey":
"6dc8ed3910b0799293799efe9a6720d6c397c6930a630c349afe7d58a1c426d80b3d81074e11dd77b8cda54069d8e0b83b900a2368ea5b48c3fdbcbfb1a36645",
"firstname": "Gao",
"card_number": "4111111111111111",
"org": "tylersoft_eclectics",
"card_expiration_year": "2025",
"Serviceid": "1010",
"secondname": "Gabo",
"card_cvNumber": "123",
"card_cardType": "001",
"processingcode": "000010",
"phone": "26774374154",
"card_expiration_month": "12",
"currency": "BWP",
"email": "ggaboiphiwe@outlook.com",
"timestamp": "2025-04-16 21:47:48"
}

Sample Response

{"statuscode":"00","tranid":"2025-05-01
09:25:41","statusmessage":"COMPLETED","id":"746457812540645060 4807","accessToken":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJqdGkiOiJjZGVhMTZiYi05NTg0LTRlMTQtOTg5ZC01M2JhZjBjMDBiZjEiLCJpYXQiOjE3NDY0NTc4MTIsImlzcyI6IjVkZDgzYmYwMGU0MjNkMTQ5OGRjYmFjYSIsImV4cCI6MTc0NjQ2MTQxMiwiT3JnVW5pdElkIjoiNjQxOWIyZjRkODdhMjUwYTQyNTExMzNkIiwiUmVmZXJlbmNlSWQiOiJlYmQzMTY2ZC0wNGFiLTQwYjgtYWJiZi0yNDcxMjYxMmFjZGMifQ.RKsEVYAd9pkm8r76kMp4Rg7JdZ5e3NHAJyPPArpE9No","deviceDataCollectionUrl":"https://centinelapistag.cardinalcommerce.com/V1/Cruise/Collect","referenceId":"ebd3166d-04ab-40b8-abbf-24712612acdc"}

URL: https://domain:port/tcp/api/deviceData-Collection

Device Data Collection is initiated on the front end after you receive the device data collection reply as described above, a hidden iframe is rendered to the browser in order to profile the customer device. Upon receiving the device collection details, Initiate a form POST in a hidden iframe and send the **accessToken** JWT to the **deviceDataCollectionUrl** URL returned by TCP

<iframe name=”ddc-iframe” height="1" width="1" style="display: none;"></iframe>
<form id="ddc-form" target=”ddc-iframe” method="POST" action="**<<deviceDataCollectionUrl>>**">
<input type="hidden" name="JWT" value="**<<accessToken>>**" />
</form>

After device data collection has completed, an event will be generated. You will need to trap this event but there is no requirement to retain any of the data items received in the event payload. The Example below shows how to subscribe to the event and add JavaScript to receive the response from the device data collection iframe. Place the JavaScript after the closing </body> element.

Note that the event.origin URL is variable depending on your environment:
- Test: https://centinelapistag.cardinalcommerce.com
- Production: https://centinelapi.cardinalcommerce.com

```
<script>
window.addEventListener("message", (event) => {
// {MessageType: "profile.completed", Session Id: "0_57f063fd-659a-
// 4779-b45b-9e456fdb7935", Status: true}
if (event.origin === "https://centinelapistag.cardinalcommerce.com") {
let data = JSON.parse(event.data);
console.log('Merchant received a message:', data);
}
if (data !== undefined && data.Status) {
console.log();
}
```

}, false);

</script>

Payment

This API function is used to complete the payment. Upon receiving the payment request from the client, TCP will respond with the payer authentication url and access token that the client would need to redirect the customer to so that they can input the OTP they received from the issuing bank. The client need to include the device data on the payment request as a backup in case there are any issues on the above device profiling process. The client should not run this service before running the device data collection call because one of the request parameters for Payment API requires the **referenceId** which was returned of the Device Data Collection response. Also the request parameters for this API call should be similar to the Device Data Collection call, but for payment there are additional parameters (Device Data)

Sample Request


```
{
 "tranid": "2025-05-01 09:25:41",
 "country": "Botswana",
 "amount": "1",
" cbyreferenceId ": " ebd3166d-04ab-40b8-abbf-24712612acdc ",
 "authkey":
"6dc8ed3910b0799293799efe9a6720d6c397c6930a630c349afe7d58a1c426d80b3d81074e11dd77b8
cda54069d8e0b83b900a2368ea5b48c3fdbcbfb1a36645",
 "firstname": "Gao",
 "card_number": "4111111111111111",
 "org": "tylersoft_eclectics",
 "card_expiration_year": "2025",
 "Serviceid": "1010",
 "secondname": "Gabo",
 "card_cvNumber": "123",
```

```
 "card_cardType": "001",
 "processingcode": "000010",
 "phone": "26774374154",
 "card_expiration_month": "12",
 "currency": "BWP",
 "email": "ggaboiphiwe@outlook.com",
 "timestamp": "2025-04-16 21:47:48",
 "ipAddress": "123.45.65.20",
 "httpAcceptContent": "test",
 "httpBrowserLanguage": "en_us",
 "httpBrowserJavaEnabled": "false",
 "httpBrowserJavaScriptEnabled": "false",
  "httpBrowserColorDepth": "24",
 "httpBrowserScreenHeight": "100000",
 "httpBrowserScreenWidth": "100000",
 "httpBrowserTimeDifference": "300",
 "userAgentBrowserValue": "GxKnLy8TFDUFxJP1t"
}
```


URL: https://domain:port/tcp/api/request

Sample Response


```
{"pareq":"eyJtZXNzYWdlVHlwZSI6IkNSZXEiLCJtZXNzYWdlVmVyc2lvbiI6IjIuMi4wIiwidGhyZWVEU1NlcnZlclRyYW5zSUQiOiI2OTFhZjI2YS00Mzk3LTQyMjAtYWM4Yy1iYTdiNWEzMTQ0YzIiLCJhY3NUcmFuc0lEIjoiMDM1OGRiMDYtYTIxYS00ZWU1LTk1YWUtODI2NTE5Y2RhZWYxIiwiY2hhbGxlbmdlV2luZG93U2l6ZSI6IjAyIn0","pastepup":"https://centinelapistag.cardinalcommerce.com/V2/Cruise/StepUp","accessToken":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJqdGkiOiI5N2VlNDAyZC05ODU2LTQzYjQtOGUzYi1jMmM3YTZiMjI4NjEiLCJpYXQiOjE3NDYyNjEyNTUsImlzcyI6IjVkZDgzYmYwMGU0MjNkMTQ5OGRjYmFjYSIsImV4cCI6MTc0NjI2NDg1NSwiT3JnVW5pdElkIjoiNjQxOWIyZjRkODdhMjUwYTQyNTExMzNkIiwiUGF5bG
```

9hZCI6eyJBQ1NVcmwiOiJodHRwczovLzBtZXJjaGFudGFjc3N0YWcuY2FyZGluYWxjb21tZXJjZS5jb20vTWVyY2hhbnRBQ1NXZWIvY3JlcS5qc3AiLCJQYXlsb2FkIjoiZXlKdFpYTnpZV2RsVkhsd1pTSTZJa05TWlhFaUxDSnRaWE56WVdkbFZtVnljMmx2YmlJNklqSXVNaTR3SWl3aWRHaHlaV1ZFVTFObGNuWmxjbFJ5WVc1elNVUWlPaUkyT1RGaFpqSTJZUzAwTXprM0xUUXlNakF0WVdNNFl5MWlZVGRpTldFek1UUTBZeklpTENKaFkzTlVjbUZ1YzBsRUlqb2lNRE0xT0dSaU1EWXRZVEl4WVMwMFpXVTFMVGsxWVdVdE9ESTJOVEU1WTJSaFpXWXhJaXdpWTJoaGJHeGxibWRsVjJsdVpHOTNVMmw2WlNJNklqQXlJbjAiLCJUcmFuc2FjdGlvbklkIjoiWGd6ZElsdUp2bElrdFBwNkNSNzAifSwiT2JqZWN0aWZ5UGF5bG9hZCI6dHJ1ZSwiUmV0dXJuVXJsIjoiaHR0cHM6Ly93d3cucGF5YmlsbHMuY28uYncvRWNvbW1lcmNlUm91dGVyL2FwaXMvcGF5ZXJfYXV0aFJlc3BvbnNlLnBocCJ9.vzlUoVUWGBvPNDyOhZsBeombsgXvs9_RrdVxyGCD2wY","message":"PENDING_AUTHENTICATION","statusCode":"001"}

If the issuing bank does not require further authentication, the transaction would be processed without having to redirect the customer to the authentication page, in this case TCP will send the final status via a callback.

If authentication is required (TCP will respond with the message PENDING_AUTHENTICATION), the client must initiate step-up authentication. This is done by creating an iframe to send a POST request to the step-up URL. The iframe manages customer interaction with the card-issuing bank's Access Control Server

Iframe Parameters

- Form POST Action: The URL posted by the iframe is the pastepup field returned by TCP responding to the payment request.

- JWT POST Parameter: Use the accessToken field returned by TCP responding to the payment request.

- MD POST Parameter: This optional merchant-defined data is returned in the response.

Example Code

The example below shows an example iFrame and form.

<iframe name=”step-up-iframe" height="400" width="400" style="display: none;"></iframe>

<form id="step-up-form" target=”step-up-iframe" method="post" action="<< pastepup **value**>>">

<input type="hidden" name="JWT" value="<<**accessToken value**>>" />

<input type="hidden" name="MD" value="<<**optional data returned as is**”/>

</form>

Final Transaction Status

The final transaction status will be delivered to the client via a callback to the URL that the client would need to provide. Upon receiving the callback, the client should reply with an acknowledgement to confirm that the callback was received.

Sample callback to the Client

{"statuscode":"100","tranid":"2025-05-01 09:25:41","status":"Request was processed successfully"}

Sample ACK reply from Client

{"statuscode":"00","tranid":"2025-05-01 09:25:41"}